TCGA Differential Gene Expression Computation

Bokang Shi

Visiting Student, Imperial College London

Cell Systems Laboratory, Institute for Protein Research, Osaka University

Summer Internship

Supervisor: Prof. Mariko Okada

Date: 9 July 2026

Data

- TCGA RNA-seq
- TCGAbiolinks
- GDC STAR Counts workflow

Workflow

1. Query TCGA project with TCGAbiolinks.
2. Download tumour and normal samples.
3. Skip projects with fewer than two tumour or two normal samples.
4. Prepare count matrix.
5. Filter low-expression genes.
6. Perform differential expression analysis using DESeq2 with the experimental design defined by sample condition (tumour vs normal).
7. Export DEG results as CSV.

Output

Each cancer type produces one CSV containing:

- gene ID
- gene symbol
- log2FoldChange
- p-value
- adjusted p-value

Current limitation

Only TCGA normal samples are currently used.

GTEx integration and batch-effect correction have not yet been implemented.

R script

C:\Users\Bokan\Documents\context-dependent-GRNs\scripts\compute_tcga_deseq2.R

```
#!/usr/bin/env Rscript

# TCGA tumor-vs-normal DESeq2 result tables.

suppressPackageStartupMessages({
  library(DESeq2)
  library(TCGAbiolinks)
  library(SummarizedExperiment)
})

parse_args <- function(args) {
  out <- list(
    projects_csv = "outputs/tcga_degs/manifest/tcga_projects.csv",
    output_dir = "outputs/tcga_degs",
    gdc_data_dir = "data/gdc",
    projects = "",
    max_projects = NA_integer_,
    workflow_type = "STAR - Counts",
    tumor_sample_types = "Primary Tumor",
    normal_sample_types = "Solid Tissue Normal",
    # Placeholder threshold for the current TCGA-only run; review before final analysis.
    min_normal = 2L,
    min_tumor = 2L,
    min_count = 10L,
    min_count_samples = 3L,
    dry_run = FALSE,
    resume = FALSE
  )

  i <- 1L
  while (i <= length(args)) {
    key <- args[[i]]
    if (key == "--dry-run") {
      out$dry_run <- TRUE
      i <- i + 1L
      next
    }
    if (key == "--resume") {
      out$resume <- TRUE
      i <- i + 1L
      next
    }
    if (i == length(args)) {
      stop(sprintf("Missing value for argument %s", key))
    }
    value <- args[[i + 1L]]
    if (key == "--projects-csv") out$projects_csv <- value
    else if (key == "--output-dir") out$output_dir <- value
    else if (key == "--gdc-data-dir") out$gdc_data_dir <- value
    else if (key == "--projects") out$projects <- value
    else if (key == "--max-projects") out$max_projects <- as.integer(value)
    else if (key == "--workflow-type") out$workflow_type <- value
    else if (key == "--tumor-sample-types") out$tumor_sample_types <- value
    else if (key == "--normal-sample-types") out$normal_sample_types <- value
    else if (key == "--min-normal") out$min_normal <- as.integer(value)
    else if (key == "--min-tumor") out$min_tumor <- as.integer(value)
    else if (key == "--min-count") out$min_count <- as.integer(value)
    else if (key == "--min-count-samples") out$min_count_samples <- as.integer(value)
    else stop(sprintf("Unknown argument: %s", key))
    i <- i + 2L
  }
  out
}

first_existing_col <- function(df, candidates) {
  for (candidate in candidates) {
    if (candidate %in% colnames(df)) return(candidate)
  }
  NA_character_
}

split_config_values <- function(value, default) {
  if (is.null(value) || length(value) == 0L || is.na(value) || !nzchar(trimws(value))) {
    value <- default
  }
  parts <- unlist(strsplit(as.character(value), "\\s*;\\s*|\\s*\\|\\s*"))
  parts <- trimws(parts)
  parts[nzchar(parts)]
```

```
}

collapse_values <- function(values) {
  values <- unique(values[!is.na(values) & nzchar(values)])
  if (length(values) == 0L) return("")
  paste(values, collapse = "; ")
}

manifest_value <- function(project_row, column, default) {
  if (column %in% names(project_row)) {
    value <- project_row[[column]]
    if (!is.na(value) && nzchar(trimws(as.character(value)))) return(as.character(value))
  }
  default
}

project_config <- function(project_row, cfg) {
  workflow <- manifest_value(project_row, "workflow_type", cfg$workflow_type)
  tumor_source <- manifest_value(project_row, "tumor_source", "GDC")
  normal_source <- manifest_value(project_row, "normal_source", "GDC")
  list(
    workflow_type = workflow,
    tumor_source = tumor_source,
    normal_source = normal_source,
    tumor_sample_types = split_config_values(
      manifest_value(project_row, "tumor_sample_types", cfg$tumor_sample_types),
      cfg$tumor_sample_types
    ),
    normal_sample_types = split_config_values(
      manifest_value(project_row, "normal_sample_types", cfg$normal_sample_types),
      cfg$normal_sample_types
    )
  )
}

select_count_assay <- function(se) {
  available <- SummarizedExperiment::assayNames(se)
  preferred <- c("unstranded", "stranded_first", "stranded_second")
  selected <- preferred[preferred %in% available]
  if (length(selected) > 0L) return(selected[[1L]])
  available[[1L]]
}

make_gene_annotation <- function(se) {
  row_data <- as.data.frame(SummarizedExperiment::rowData(se))
  gene_id_col <- first_existing_col(row_data, c("gene_id", "ensembl_gene_id", "ensembl_id"))
  gene_symbol_col <- first_existing_col(row_data, c("gene_name", "external_gene_name", "gene_symbol"))
  gene_type_col <- first_existing_col(row_data, c("gene_type", "gene_biotype", "external_gene_type"))

  gene_id <- rownames(se)
  if (!is.na(gene_id_col)) gene_id <- as.character(row_data[[gene_id_col]])

  gene_symbol <- rep(NA_character_, nrow(row_data))
  if (!is.na(gene_symbol_col)) gene_symbol <- as.character(row_data[[gene_symbol_col]])

  gene_type <- rep(NA_character_, nrow(row_data))
  if (!is.na(gene_type_col)) gene_type <- as.character(row_data[[gene_type_col]])

  data.frame(
    gene_id = gene_id,
    gene_symbol = gene_symbol,
    gene_type = gene_type,
    stringsAsFactors = FALSE
  )
}

discover_gdc_metadata <- function(project_id) {
  query <- GDCquery(
    project = project_id,
    data.category = "Transcriptome Profiling",
    data.type = "Gene Expression Quantification"
  )
  getResults(query)
}

metadata_profile <- function(metadata) {
  if (is.null(metadata) || nrow(metadata) == 0L) {
    return(list(
      sample_type_col = NA_character_,
      workflow_col = NA_character_,
      sample_types_found = character(),
      workflow_types_found = character()
    ))
  }
  sample_type_col <- first_existing_col(metadata, c("sample_type", "cases.samples.sample_type"))
```

```
  workflow_col <- first_existing_col(metadata, c("analysis_workflow_type", "workflow_type"))
  sample_types <- character()
  workflow_types <- character()
  if (!is.na(sample_type_col)) sample_types <- sort(unique(as.character(metadata[[sample_type_col]])))
  if (!is.na(workflow_col)) workflow_types <- sort(unique(as.character(metadata[[workflow_col]])))
  list(
    sample_type_col = sample_type_col,
    workflow_col = workflow_col,
    sample_types_found = sample_types,
    workflow_types_found = workflow_types
  )
}

diagnose_metadata <- function(metadata, profile, cfg_row) {
  if (is.null(metadata) || nrow(metadata) == 0L) {
    return("no_gdc_expression_quantification_files")
  }
  if (is.na(profile$sample_type_col)) return("sample_type_column_not_found")
  if (is.na(profile$workflow_col)) return("workflow_type_column_not_found")
  if (!(cfg_row$workflow_type %in% profile$workflow_types_found)) {
    return(sprintf("requested_workflow_unavailable: %s", cfg_row$workflow_type))
  }
  missing_tumor <- setdiff(cfg_row$tumor_sample_types, profile$sample_types_found)
  if (length(missing_tumor) > 0L) {
    return(sprintf("requested_tumor_sample_type_unavailable: %s", collapse_values(missing_tumor)))
  }
  if (toupper(cfg_row$normal_source) != "GDC") {
    return(sprintf("external_normal_source_not_supported_in_this_script: %s", cfg_row$normal_source))
  }
  missing_normal <- setdiff(cfg_row$normal_sample_types, profile$sample_types_found)
  if (length(missing_normal) > 0L) {
    return(sprintf("requested_normal_sample_type_unavailable: %s", collapse_values(missing_normal)))
  }
  ""
}

filter_metadata_for_config <- function(metadata, profile, cfg_row) {
  if (is.null(metadata) || nrow(metadata) == 0L ||
      is.na(profile$sample_type_col) || is.na(profile$workflow_col)) {
    return(metadata[0, , drop = FALSE])
  }
  sample_types <- unique(c(cfg_row$tumor_sample_types, cfg_row$normal_sample_types))
  metadata[
    metadata[[profile$workflow_col]] == cfg_row$workflow_type &
      metadata[[profile$sample_type_col]] %in% sample_types,
    ,
    drop = FALSE
  ]
}

count_samples <- function(metadata, profile, cfg_row) {
  if (is.null(metadata) || nrow(metadata) == 0L || is.na(profile$sample_type_col)) {
    return(list(n_tumor = 0L, n_normal = 0L))
  }
  list(
    n_tumor = sum(metadata[[profile$sample_type_col]] %in% cfg_row$tumor_sample_types, na.rm = TRUE),
    n_normal = sum(metadata[[profile$sample_type_col]] %in% cfg_row$normal_sample_types, na.rm = TRUE)
  )
}

make_gdc_query <- function(project_id, cfg_row) {
  GDCquery(
    project = project_id,
    data.category = "Transcriptome Profiling",
    data.type = "Gene Expression Quantification",
    workflow.type = cfg_row$workflow_type,
    sample.type = unique(c(cfg_row$tumor_sample_types, cfg_row$normal_sample_types))
  )
}

summary_fields <- c(
  "tcga_abbreviation", "project_id", "run_mode", "status",
  "tumor_source", "normal_source",
  "configured_tumor_sample_types", "configured_normal_sample_types",
  "selected_workflow_type", "sample_types_found", "workflow_types_found",
  "n_tumor", "n_normal", "n_genes_tested", "output_file",
  "skip_reason", "message"
)

write_empty_summary <- function(path) {
  empty <- as.data.frame(setNames(replicate(length(summary_fields), character(), simplify = FALSE),
summary_fields))
  write.csv(empty, path, row.names = FALSE)
}

```

```
read_resume_summary <- function(path) {
  if (!file.exists(path)) {
    return(as.data.frame(setNames(replicate(length(summary_fields), character(), simplify = FALSE),
summary_fields)))
  }
  existing <- read.csv(path, stringsAsFactors = FALSE, check.names = FALSE)
  for (field in summary_fields) {
    if (!field %in% names(existing)) existing[[field]] <- NA_character_
  }
  existing[, summary_fields, drop = FALSE]
}

find_resume_row <- function(existing_summary, project_row) {
  if (nrow(existing_summary) == 0L) return(NULL)
  matches <- existing_summary[
    existing_summary$tcga_abbreviation == project_row$tcga_abbreviation[[1L]] &
      existing_summary$project_id == project_row$project_id[[1L]],
    ,
    drop = FALSE
  ]
  if (nrow(matches) == 0L) return(NULL)
  row <- matches[nrow(matches), , drop = FALSE]
  reusable_status <- row$status %in% c("completed", "skipped", "dry_run_ready")
  output_ok <- !identical(row$status[[1L]], "completed") ||
    (nzchar(row$output_file[[1L]]) && file.exists(row$output_file[[1L]]))
  if (isTRUE(reusable_status) && isTRUE(output_ok)) row else NULL
}

summary_row_df <- function(result) {
  normalized <- lapply(summary_fields, function(field) {
    value <- result[[field]]
    if (is.null(value) || length(value) == 0L) return(NA)
    value[[1L]]
  })
  names(normalized) <- summary_fields
  as.data.frame(normalized, stringsAsFactors = FALSE)
}

base_result <- function(tcga_abbreviation, project_id, cfg_row, profile, cfg) {
  list(
    tcga_abbreviation = tcga_abbreviation,
    project_id = project_id,
    run_mode = ifelse(isTRUE(cfg$dry_run), "dry_run", "full"),
    status = "",
    tumor_source = cfg_row$tumor_source,
    normal_source = cfg_row$normal_source,
    configured_tumor_sample_types = collapse_values(cfg_row$tumor_sample_types),
    configured_normal_sample_types = collapse_values(cfg_row$normal_sample_types),
    selected_workflow_type = cfg_row$workflow_type,
    sample_types_found = collapse_values(profile$sample_types_found),
    workflow_types_found = collapse_values(profile$workflow_types_found),
    n_tumor = NA_integer_,
    n_normal = NA_integer_,
    n_genes_tested = NA_integer_,
    output_file = "",
    skip_reason = "",
    message = ""
  )
}

compute_project <- function(project_row, cfg) {
  tcga_abbreviation <- project_row$tcga_abbreviation[[1L]]
  project_id <- project_row$project_id[[1L]]
  cfg_row <- project_config(project_row, cfg)

  message(sprintf("[%s] Discovering GDC metadata", project_id))
  metadata <- discover_gdc_metadata(project_id)
  profile <- metadata_profile(metadata)
  result <- base_result(tcga_abbreviation, project_id, cfg_row, profile, cfg)

  selected_metadata <- filter_metadata_for_config(metadata, profile, cfg_row)
  counts <- count_samples(selected_metadata, profile, cfg_row)
  result$n_tumor <- counts$n_tumor
  result$n_normal <- counts$n_normal

  diagnostic <- diagnose_metadata(metadata, profile, cfg_row)
  if (nzchar(diagnostic)) {
    result$status <- "skipped"
    result$skip_reason <- diagnostic
    result$message <- "Configured workflow/sample types are not available in discovered GDC metadata"
    return(result)
  }

  if (counts$n_tumor < cfg$min_tumor) {
    result$status <- "skipped"
```

```
    result$skip_reason <- sprintf("insufficient_tumor_samples: n_tumor=%s, min_tumor=%s", counts$n_tumor, cfg$min_tumor)
    result$message <- "DESeq2 was not run"
    return(result)
  }
  if (counts$n_normal < cfg$min_normal) {
    result$status <- "skipped"
    result$skip_reason <- sprintf("no_sufficient_tcga_normal_samples: n_normal=%s, min_normal=%s", counts$n_normal, cfg$min_normal)
    result$message <- "DESeq2 was not run"
    return(result)
  }

  if (cfg$dry_run) {
    result$status <- "dry_run_ready"
    result$message <- sprintf("Ready for DESeq2 download/run with %s files", nrow(selected_metadata))
    return(result)
  }

  query <- make_gdc_query(project_id, cfg_row)
  GDCdownload(query, method = "api", files.per.chunk = 20, directory = cfg$gdc_data_dir)
  se <- GDCprepare(query, directory = cfg$gdc_data_dir)

  col_data <- as.data.frame(SummarizedExperiment::colData(se))
  sample_type_col <- first_existing_col(col_data, c("sample_type", "definition"))
  if (is.na(sample_type_col)) {
    stop("Could not find sample type column in prepared GDC object")
  }

  requested_sample_types <- unique(c(cfg_row$tumor_sample_types, cfg_row$normal_sample_types))
  keep_samples <- col_data[[sample_type_col]] %in% requested_sample_types
  se <- se[, keep_samples]
  col_data <- as.data.frame(SummarizedExperiment::colData(se))
  condition <- ifelse(col_data[[sample_type_col]] %in% cfg_row$normal_sample_types, "normal", "tumor")
  condition <- factor(condition, levels = c("normal", "tumor"))

  n_tumor <- sum(condition == "tumor")
  n_normal <- sum(condition == "normal")
  if (n_tumor < cfg$min_tumor || n_normal < cfg$min_normal) {
    stop(sprintf("Insufficient samples after preparation: n_tumor=%s, n_normal=%s", n_tumor, n_normal))
  }

  assay_name <- select_count_assay(se)
  counts_matrix <- SummarizedExperiment::assay(se, assay_name)
  storage.mode(counts_matrix) <- "integer"

  sample_df <- data.frame(condition = condition, row.names = colnames(counts_matrix))
  dds <- DESeqDataSetFromMatrix(
    countData = counts_matrix,
    colData = sample_df,
    design = ~ condition
  )

  keep_genes <- rowSums(DESeq2::counts(dds) >= cfg$min_count) >= cfg$min_count_samples
  dds <- dds[keep_genes, ]
  dds <- DESeq(dds)

  res <- results(dds, contrast = c("condition", "tumor", "normal"))
  res_df <- as.data.frame(res)
  annotation <- make_gene_annotation(se)[keep_genes, , drop = FALSE]
  out_df <- cbind(annotation, res_df)

  out_df$tcga_abbreviation <- tcga_abbreviation
  out_df$project_id <- project_id
  out_df$n_tumor <- n_tumor
  out_df$n_normal <- n_normal

  ordered_cols <- c(
    "tcga_abbreviation", "project_id", "gene_id", "gene_symbol", "gene_type",
    "baseMean", "log2FoldChange", "lfcSE", "stat", "pvalue", "padj",
    "n_tumor", "n_normal"
  )
  out_df <- out_df[, ordered_cols]

  project_out_dir <- file.path(cfg$output_dir, "deseq2_results", tcga_abbreviation)
  dir.create(project_out_dir, recursive = TRUE, showWarnings = FALSE)
  output_file <- file.path(project_out_dir, "deseq2_results.csv")
  write.csv(out_df, output_file, row.names = FALSE)

  result$status <- "completed"
  result$n_tumor <- n_tumor
  result$n_normal <- n_normal
  result$n_genes_tested <- nrow(out_df)
  result$output_file <- normalizePath(output_file, winslash = "\\", mustWork = FALSE)
  result$message <- sprintf("Assay=%s", assay_name)
```

```
  result
}

main <- function() {
  cfg <- parse_args(commandArgs(trailingOnly = TRUE))
  dir.create(file.path(cfg$output_dir, "manifest"), recursive = TRUE, showWarnings = FALSE)
  dir.create(cfg$gdc_data_dir, recursive = TRUE, showWarnings = FALSE)

  projects <- read.csv(cfg$projects_csv, stringsAsFactors = FALSE, check.names = FALSE)
  required <- c("tcga_abbreviation", "project_id")
  missing <- setdiff(required, colnames(projects))
  if (length(missing) > 0L) {
    stop(sprintf("Project manifest is missing columns: %s", paste(missing, collapse = ", ")))
  }

  if (nzchar(cfg$projects)) {
    selected <- trimws(strsplit(cfg$projects, ",")[[1L]])
    projects <- projects[projects$tcga_abbreviation %in% selected | projects$project_id %in% selected, ]
  }
  if (!is.na(cfg$max_projects)) {
    projects <- head(projects, cfg$max_projects)
  }
  if (nrow(projects) == 0L) {
    stop("No projects selected")
  }

  summary_path <- file.path(cfg$output_dir, "manifest", "tcga_deg_summary.csv")
  existing_summary <- read_resume_summary(summary_path)
  if (!cfg$resume) {
    write_empty_summary(summary_path)
    existing_summary <- read_resume_summary(summary_path)
  }
  summary_rows <- list()

  for (i in seq_len(nrow(projects))) {
    project_row <- projects[i, , drop = FALSE]
    project_id <- project_row$project_id[[1L]]
    resume_row <- if (cfg$resume) find_resume_row(existing_summary, project_row) else NULL
    if (!is.null(resume_row)) {
      message(sprintf("[%s] resume: reusing existing %s row", project_id, resume_row$status[[1L]]))
      summary_rows[[length(summary_rows) + 1L]] <- resume_row
      summary_df <- do.call(rbind, summary_rows)
      write.csv(summary_df, summary_path, row.names = FALSE)
      next
    }
    result <- tryCatch(
      compute_project(project_row, cfg),
      error = function(e) {
        cfg_row <- project_config(project_row, cfg)
        profile <- list(sample_types_found = character(), workflow_types_found = character())
        out <- base_result(project_row$tcga_abbreviation[[1L]], project_id, cfg_row, profile, cfg)
        out$status <- "error"
        out$skip_reason <- "runtime_error"
        out$message <- conditionMessage(e)
        out
      }
    )
    summary_rows[[length(summary_rows) + 1L]] <- summary_row_df(result)
    summary_df <- do.call(rbind, summary_rows)
    write.csv(summary_df, summary_path, row.names = FALSE)
    message(sprintf("[%s] %s: %s", project_id, result$status, result$message))
  }

  message(sprintf("Wrote summary: %s", normalizePath(summary_path, winslash = "\\", mustWork = FALSE)))
}

main()
```